JavaScript Demonstration of Collections, Arrays and Objects

Below is a complete, copy-and-paste-ready example consisting of three files:

- **`index.html`** – the page structure
- **`style.css`** – simple styling to make the demo look tidy
- **`script.js`** – the heart of the demo, showing JavaScript **Objects**, **Arrays**, and a **Collection** (an array of objects)

📁 Project Structure
- `index.html` – entry point
- `style.css` – visual styling
- `script.js` – logic & data-structure demo

1️⃣ `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JS Collections, Arrays & Objects Demo</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <header>
        <h1>Collections, Arrays & Objects</h1>
        <p>A simple demonstration of core JavaScript data structures.</p>
    </header>

    <main>
        <section id="object-section">
            <h2>Object Example</h2>
```

```
        <pre id="object-output"></pre>
      </section>

      <section id="array-section">
        <h2>Array Example</h2>
        <pre id="array-output"></pre>
      </section>

      <section id="collection-section">
        <h2>Collection (Array of Objects)</h2>
        <pre id="collection-output"></pre>
      </section>
    </main>

    <script src="script.js"></script>
</body>
</html>
```

2️⃣ style.css

```
/* style.css – simple, clean layout for the demo */
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color #f4f4f9;
    color: #333;
}

header {
    background: #2c3e50;
    color: #ecf0f1;
    padding: 2rem;
    text-align: center;
}

main {
    max-width: 800px;
    margin: 2rem auto;
    padding: 1rem;
}
```

```
section {
    margin-bottom: 2rem;
}

pre {
    background: #fff;
    border: 1px solid #ddd;
    padding: 1rem;
    overflow-x: auto;
    border-radius: 4px;
}
```

3️⃣ script.js

```
/* script.js – demonstrates Objects, Arrays, and a Collection (array of objects) */

// ---------------------------------------------------------------
// 1️⃣ Object Example
// ---------------------------------------------------------------
// A simple "user" object with properties and a method.
const user = {
    firstName: "Alice",
    lastName:  "Johnson",
    age:       30,
    fullName: function() {
        return `${this.firstName} ${this.lastName}`;
    }
};

// Display the object and its method result in the HTML.
const objectOutput = document.getElementById("object-output");
objectOutput.textContent = `
User object:
- firstName: ${user.firstName}
- lastName:  ${user.lastName}
- age:       ${user.age}
- fullName(): ${user.fullName()}
`;

// ---------------------------------------------------------------
// 2️⃣ Array Example
// ---------------------------------------------------------------
```

```
// An array of numbers, with common array methods used to transform it.
const numbers = [12, 4, 57, 23, 9, 71];

// Map: square each number
const squared = numbers.map(n => n * n);

// Filter: keep only even numbers
const evens = numbers.filter(n => n % 2 === 0);

// Reduce: sum all numbers
const total = numbers.reduce((acc, n) => acc + n, 0);

const arrayOutput = document.getElementById("array-output");
arrayOutput.textContent = `
Original array: [${numbers.join(", ")}]

Squared (map):        [${squared.join(", ")}]

Even numbers only (filter): [${evens.join(", ")}]

Sum of all numbers (reduce): ${total}
`;

// ---------------------------------------------------------------
// 3️⃣ Collection Example  (Array of Objects)
// ---------------------------------------------------------------
// A "collection" of product objects. This mimics a real-world data set.
const products = [
   { id: 1, name: "Laptop",   price: 999, inStock: true },
   { id: 2, name: "Mouse",    price: 25,  inStock: false },
   { id: 3, name: "Keyboard",price: 75,  inStock: true },
   { id: 4, name: "Monitor",  price: 199, inStock: true }
];

// 3a) Access a specific item by index
const firstProduct = products[0];

// 3b) Map over the collection to get just the names
const productNames = products.map(p => p.name);

// 3c) Filter to only in-stock items
const inStockProducts = products.filter(p => p.inStock);

// 3d) Reduce to calculate total inventory value (price * quantity assumed 1)
```

```
const totalValue = products.reduce((sum, p) => sum + p.price, 0);

const collectionOutput = document.getElementById("collection-output");
collectionOutput.textContent = `
Collection of products (array of objects):

• First product: ID=${firstProduct.id}, Name="${firstProduct.name}", Price=$${firstProduct.price}, InStock=${firstProduct.inStock}

• All product names: [${productNames.join(", ")}]

• In-stock products: [${inStockProducts.map(p => p.name).join(", ")}]

• Total value of all products (assuming qty=1): $${totalValue}
`;

// ---------------------------------------------------------------
// Optional: a tiny demo of “adding” to the collection dynamically
// ---------------------------------------------------------------
/*
products.push({ id: 5, name: "Webcam", price: 45, inStock: true });
console.log("Updated collection:", products);
*/
```

How to Run

1. Save the three files in the **same folder**: `index.html`, `style.css`, `script.js`.
2. Open `index.html` in any modern web browser.
3. The page will show three sections populated with the object, array, and collection data, plus the results of common array methods.

Key concepts illustrated

Concept	What the code shows
Object	`user` object with properties + a method (`fullName`).
Array	`numbers` array demonstrating `map`, `filter`, and `reduce`.

Collection	`products` array of objects, showing indexing, `map`, `filter`, and `reduce` on a real-world data set.

Feel free to extend the demo—add more objects, experiment with nested structures, or wire up UI controls to modify the collections on the fly. Happy coding!